

Machine learning : Déchets

Prédire la production de déchets ménagers par habitant et par département

MEMBRES DU GROUPE

ALVES Rudy

BENASSIE Noé

TAVERNIER Florian

ANTHONY Josselin

DATE

24 septembre 2026

DÉPÔT GITHUB

github.com/Rxdy/ML-DMA

Données : ADEME (SINOE) et INSEE, data.gouv.fr, licence Ouverte 2.0

Sommaire

Introduction	3
1. Choix du thème : les déchets ménagers	4
2. La source principale : SINOE (ADEME)	5
3. Consolider les données	6
4. Qu'allons-nous prédire ?	7
5. Nettoyage des données	9
6. Se débarrasser des données inutiles	10
7. Les entrées X et la sortie y	11
8. Mettre les données en forme	12
9. Établir la baseline	13
10. Premiers modèles	14
11. Preprocessor et pipeline	16
12. Évaluation et interprétation	17
13. Tester le modèle et prédire 2025	20
14. Comment améliorer le modèle	21
15. Problèmes rencontrés et conclusion	25
Annexe A. Clustering des profils de traitement	26
Annexe B. Détail des calculs	27
Annexe C. Guide d'utilisation du projet	30

Introduction

Ce rapport retrace, dans l'ordre où nous les avons prises, les décisions du projet : le choix du thème, la recherche et la consolidation des données, leur nettoyage, le choix de ce que l'on prédit, puis la construction et l'évaluation des modèles. Pour chaque étape, nous indiquons ce que nous avons fait et **pourquoi**.

La question posée : peut-on prédire combien de kilos de déchets ménagers chaque habitant d'un département produira lors de la prochaine enquête, à partir de ce que l'on sait des enquêtes précédentes ?

Le résultat en trois phrases

La production de déchets par habitant d'un département est très stable d'une enquête à l'autre : la simple règle « comme à l'enquête précédente » explique déjà 70 % à 87 % des écarts entre départements. Nos modèles (régression linéaire, Random Forest) font aussi bien, mais pas mieux de façon utile. Avec les variables disponibles, l'historique du département contient presque toute l'information.

Le code, les données et le journal de bord détaillé sont disponibles sur le dépôt github.com/Rxdy/ML-DMA. Chaque étape correspond à un script numéroté du dossier `dechets/scripts/`.

Chronologie en un coup d'œil

Étape	Ce qui a été fait	Script
1	Choix du thème : les déchets ménagers et assimilés	—
2	Source principale ADEME (SINOE) : exploration	01_explore.py
3	Consolidation avec d'autres sources (population, 2023, revenu)	07_income_prep.py
4	Choix de ce que l'on prédit : le ratio de déchets par habitant	03_regression_prep.py
5	Nettoyage : formats, clés, valeurs manquantes, doublons	01, 03
6	Suppression des données inutiles	03, 05
7	Les entrées X et la sortie y du modèle	09_pipeline.py
8	Mise en forme : profil de traitement, historique, découpage train/test	02 à 05
9	Baseline : le repère à battre	06_train_model.py
10	Premiers modèles, courbe d'apprentissage, test du revenu	06, 08
11	Preprocessor et pipeline	09_pipeline.py
12	Évaluation détaillée et interprétation	09, 10
13	Prédiction de l'enquête 2025	12_prediction_2025.py
14	Comment améliorer le modèle : analyse des erreurs et pistes	11_analyse_erreurs.py

ÉTAPE 1 • 16 septembre 2026

1. Choix du thème : les déchets ménagers

Les **déchets ménagers et assimilés (DMA)** sont les déchets collectés par le service public de gestion des déchets : ceux des ménages, et ceux des petites activités économiques (commerces, artisans) collectés avec eux. Dans nos données, ils sont répartis en 7 catégories :

- ordures ménagères résiduelles (la poubelle « classique ») ;
- matériaux recyclables (emballages, papiers, verre) ;
- déchets verts et biodéchets ;
- encombrants ;
- déchets dangereux (y compris les équipements électriques et électroniques) ;
- déblais et gravats ;
- autres.

Chaque tonne est ensuite traitée de l'une de 7 façons : incinération avec ou sans récupération d'énergie, valorisation matière (recyclage), valorisation organique (compostage, méthanisation), stockage, stockage pour déchets inertes, ou traitement non précisé.

Pourquoi ce thème

- **Un enjeu concret** : chaque territoire organise sa collecte et son traitement, et la réduction des déchets fait l'objet d'objectifs publics. Anticiper les volumes a une utilité réelle.
- **Des données ouvertes et officielles** : l'ADEME publie ces chiffres pour tous les départements français depuis 2009, sous licence ouverte.
- **Une question de prédiction naturelle** : combien de déchets produira un département ? La réponse est un nombre, connu pour les années passées, ce qui en fait un problème d'apprentissage supervisé.
- **Des ordres de grandeur parlants** : environ 38 à 41 millions de tonnes par an en France (gravats compris), soit un peu plus de 500 kg par habitant et par an en moyenne hors gravats.

ÉTAPE 2 • 16 septembre 2026

2. La source principale : SINOE (ADEME)

Notre point de départ est le jeu de données « **Destination des DMA collectés par type de traitement** » de l'ADEME, issu de sa base SINOE et publié sur data.gouv.fr (licence Ouverte 2.0) : fichier `sinoe_dma.csv`.

Caractéristique	Valeur
Taille	14 660 lignes × 10 colonnes
Format	« long » : une ligne = année × département × type de déchet × type de traitement tonnage
Couverture	101 départements (métropole et outre-mer), 14 régions
Période	2009 à 2021, années impaires uniquement (7 enquêtes)
Qualité	aucune valeur manquante, aucun doublon, 54 tonnages nuls, aucun négatif

Ce qui était intéressant

- **Le niveau de détail** : pour chaque département, on sait quelle part des déchets part en incinération, en recyclage, en stockage... Cela permet de décrire la **politique de traitement** de chaque territoire, et de regrouper les départements qui se ressemblent.
- **La profondeur temporelle** : 7 enquêtes sur 13 ans pour chaque département.

Ce qui posait question

- **Seulement des années impaires, et rien après 2021**, alors que la page data.gouv.fr annonçait une mise à jour récente. Vérification faite sur les quatre jeux SINOE de l'ADEME : tous indiquent « années impaires à partir de 2009 ». L'enquête est **biennale par construction** : ce n'est pas une donnée manquante à combler. La date de mise à jour correspond à une republication du fichier.
- **Pas de population** : impossible de comparer un grand et un petit département, ni de raisonner par habitant avec ce seul fichier.
- **Pas de 2023** : le fichier s'arrête à l'enquête 2021.

Ces deux manques ont motivé l'étape suivante : consolider avec d'autres sources.

ÉTAPE 3 · 16 septembre 2026

3. Consolider les données

Nous avons cherché d'autres jeux publiés par l'ADEME et l'INSEE, avec une contrainte : pouvoir les relier proprement à nos départements par le code département (C_DEPT).

Fichier	Lignes	Période	Apport	Décision
sinoe_dma.csv	14 660	2009-2021	Tonnage par type de déchet et de traitement	Clustering
chiffres_cles_hors_gravats	800	2009-2023	Tonnage, population et ratio kg/habitant déjà calculés. Seule source avec 2023	Base de la régression
chiffres_cles_avec_gravats	701	2009-2021	Mêmes indicateurs, gravats compris	Contrôle, puis écarté
insee_population (âge, sexe)	101 dép.	1975-2023	Pyramide des âges par département	Non utilisé
insee_revenu_median	491	2013-2021	Revenu médian (Filosofi), assemblé à partir de 5 formats différents	Testé (étape 10)

Pourquoi le fichier « hors gravats » comme base

- Il apporte la **population** et le **ratio par habitant**, ce qui manquait au fichier principal.
- C'est le **seul qui va jusqu'en 2023** : 8 enquêtes au lieu de 7.
- Les gravats viennent surtout des chantiers (BTP), pas du comportement des ménages. Ils ajoutent de la variabilité sans rapport avec la population : l'écart-type du ratio passe de 80,3 kg/hab sans gravats à 100,6 kg/hab avec.

Vérifications de cohérence entre sources

- Les deux fichiers « chiffres-clés » donnent **exactement la même population** pour chaque département et chaque année (0 écart sur 701 lignes comparées).
- Les codes et noms de départements et de régions sont identiques d'un fichier à l'autre.
- **Piège évité** : la colonne TONNAGE_DMA porte le même nom dans deux fichiers mais pas le même périmètre. En additionnant le détail de sinoe_dma.csv, on trouve 10,5 % d'écart en moyenne avec le fichier hors gravats (jusqu'à 28 % dans les Landes). En retirant les gravats du calcul, l'écart tombe à 0,03 %. La cible de la régression est donc toujours lue dans le fichier hors gravats, jamais recalculée.

Et le revenu des ménages ?

Plus tard dans le projet, pour tenter d'améliorer les prédictions, nous avons cherché une variable explicative supplémentaire. Kaggle ne proposait que des jeux d'images de déchets ou des données mondiales sans lien avec les départements français. Nous nous sommes tournés vers l'INSEE (revenu médian Filosofi), publié dans un format différent chaque année : 5 fichiers ont dû être harmonisés (script 07_income_prep.py).

ÉTAPE 4 · 16 septembre 2026

4. Qu'allons-nous prédire ?

Avant d'aller plus loin dans le nettoyage, il fallait fixer l'objectif : c'est lui qui détermine quelles données sont vraiment nécessaires. Trois options ont été envisagées :

Option	Idée	Décision
1	Régression du ratio de déchets par habitant, par département et par année, plus un clustering des départements selon leur profil de traitement	Retenue
2	Classification du mode de traitement dominant de chaque département	Écartée
3	Séries temporelles : extrapoler au-delà de 2023	Écartée : 8 points par département, trop peu

La cible : le ratio par habitant, pas le tonnage

La corrélation entre le tonnage d'un département et sa population vaut **0,97**. Prédire le tonnage brut reviendrait à réapprendre une multiplication (ratio × population) : le modèle aurait l'air excellent sans avoir rien appris d'utile. C'est une **fuite de données**. En prédisant `RATIO_DMA` (kg par habitant), on retire l'effet de taille, et le modèle doit expliquer les vraies différences de comportement entre départements.

Ce que disent les corrélations avec la cible

Variable	Corrélation	Lecture
Ratio à l'enquête précédente	0,91	Signal très fort : un département change peu en deux ans
Population	0,17	Signal faible : les départements peuplés produisent un peu moins par habitant
Profil de traitement (cluster)	0,13	Signal faible
Année	0,02	Quasi nul : il n'y a pas de tendance nationale nette

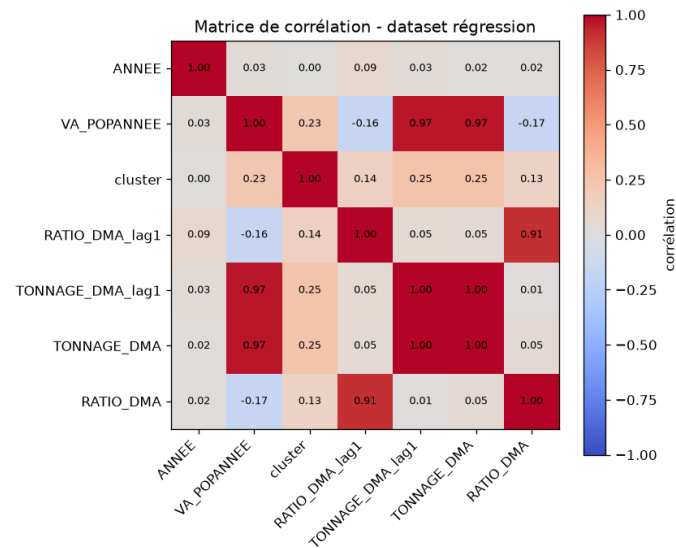


Figure 1 — Matrice de corrélation des variables du jeu de régression (script 04_correlation.py).

Conséquence directe : le temps sera représenté par **la valeur de l'enquête précédente** du même département, pas par le numéro de l'année. C'est un problème de **régression supervisée** : chaque exemple est un couple département × année dont on connaît la réponse.

ÉTAPE 5 · 16 septembre 2026

5. Nettoyage des données

Chaque opération répond à un constat de l'exploration, aucune n'est appliquée « par défaut ».

Constat	Traitement	Pourquoi
Les fichiers « chiffres-clés » utilisent ; et la virgule décimale	Lecture avec <code>sep=' ; ', decimal=', ' et encodage UTF-8 avec BOM</code>	Sinon les nombres sont lus comme du texte
Codes département de longueur variable (1 / 01)	Codes complétés à 2 caractères	Sinon la jointure perd des départements sans message d'erreur
Aucune valeur manquante dans les sources	Rien à imputer	—
54 tonnages à 0, aucun négatif	Conservés	Absence réelle de tonnage pour une combinaison donnée, pas une erreur
Années paires absentes	Non imputées	L'enquête est biennale : il n'y a rien à combler
Doublons possibles	Contrôle à deux niveaux (voir ci-dessous)	Un doublon fausserait l'historique

Les doublons : contrôlés à deux niveaux

- **Lignes strictement identiques** : 0.
- **Clés répétées**, le cas le plus dangereux : une même combinaison déclarée deux fois avec des valeurs différentes, qu'un simple `uplicated()` ne détecte pas. Clé année × département × type de déchet × type de traitement pour le fichier principal, département × année pour les autres : 0 doublon.
- Un même département apparaît bien une fois par enquête. Ce n'est pas un doublon : ce sont des observations successives du même territoire (données de panel).

Pourquoi c'est important ici

Une clé en double casserait la variable d'historique : « l'enquête précédente » serait la ligne dupliquée elle-même, c'est-à-dire la réponse donnée au modèle. Une jointure sur une clé en double multiplierait aussi les lignes en silence. Ces contrôles sont inscrits dans le code : unicité vérifiée par `assert`, jointures déclarées avec `validate='many_to_one'` et nombre de lignes comparé avant et après.

ÉTAPE 6 · 16 septembre 2026

6. Se débarrasser des données inutiles

Garder une donnée « au cas où » alourdit le modèle et peut l'induire en erreur. Voici ce qui a été retiré, et pourquoi.

Élément écarté	Raison
L_REGION, C_REGION, N_DEPT	Libellés et doublons de codes : ils identifient un département, ils ne l'expliquent pas. C_DEPT est gardé comme clé de jointure uniquement.
ANNEE comme variable d'entrée	Corrélation de 0,02 avec la cible. Remplacée par l'historique du département.
TONNAGE_DMA comme cible	Quasi-copie de la population (corrélation 0,97) : fuite de données.
Fichier « avec gravats »	Sert au contrôle croisé de la population, puis écarté : les gravats brouillent la comparaison par habitant.
Fichier INSEE population par âge et sexe	Redondant avec la population déjà présente dans les fichiers ADEME.
Les 101 lignes de 2009	Première enquête : pas d'enquête précédente, donc pas d'historique possible.

Après ce tri, le jeu de données de régression compte **699 lignes** (101 départements × 7 enquêtes de 2011 à 2023, moins quelques trous ponctuels de l'enquête) et **4 variables d'entrée** :

Variable	Rôle	Nature
RATIO_DMA_lag1	Ratio de l'enquête précédente, même département	Numérique
TONNAGE_DMA_lag1	Tonnage de l'enquête précédente	Numérique
VA_POPANNEE	Population du département	Numérique
c_luster	Profil de traitement du département (étape 8)	Catégorielle
RATIO_DMA	Cible : kg de déchets par habitant	Numérique

ÉTAPE 7 · 16 septembre 2026

7. Les entrées X et la sortie y

Une fois les données nettoyées et triées, on les sépare en deux, selon la convention de l'apprentissage supervisé :

- **X, les données d'entrée** (les « features ») : ce que le modèle connaît au moment de prédire. C'est un tableau : une ligne par exemple, une colonne par variable.
- **y, la donnée de sortie** (la « cible ») : ce que le modèle doit apprendre à prédire. C'est une seule colonne, avec une valeur par ligne de X.

Apprendre, c'est trouver une fonction **f** telle que **f(X)** y sur les exemples connus, puis l'appliquer à des X dont on ne connaît pas encore le y.

Dans notre projet

	Contenu	Dimensions
X	VA_POPANNEE, RATIO_DMA_lag1, TONNAGE_DMA_lag1, cluster	699 lignes × 4 colonnes
y	RATIO_DMA : kg de déchets par habitant	699 valeurs
X_train, y_train	Enquêtes 2011 à 2019 : le modèle apprend dessus	501 × 4, et 501 valeurs
X_test, y_test	Enquêtes 2021 et 2023 : on compare les prédictions à y_test	198 × 4, et 198 valeurs

Exemple réel : trois lignes de X_train et leur y (enquête 2019)

Département	VA_POPANNEE	RATIO_DMA_lag1	TONNAGE_DMA_lag1	cluster	y = RATIO_DMA
Ain	652 432	461,9	297 186	1	471,3
Landes	413 690	834,0	339 795	3	886,6
Paris	2 165 423	477,3	1 044 088	1	473,3

Pour l'Ain en 2019, le modèle reçoit la population, le ratio et le tonnage de l'enquête précédente (2017) et le profil de traitement 1 (incinération). Il doit en déduire y, le ratio 2019. Une ligne = un département à une date ; C_DEPT et ANNEE servent à construire et à découper les données, mais ne font pas partie de X.

Pourquoi ce découpage compte

Toute information présente dans X doit être connue **avant** la date prédite. C'est pour cela que X contient le ratio de l'enquête **précédente** et pas celui de l'année prédite : sinon, on donnerait la réponse au modèle. Après le preprocessor (étape 11), X passe de 4 à 7 colonnes, le cluster étant éclaté en 4 colonnes 0/1.

ÉTAPE 8 · 16 septembre 2026

8. Mettre les données en forme

Profil de traitement et clustering

Pour chaque département, le tonnage est réparti entre les 7 modes de traitement, puis converti en pourcentages (chaque ligne somme à 100). Sans cette mise en pourcentage, un grand département ressemblerait à un autre grand département simplement par sa taille. Un k-means regroupe ensuite les départements aux profils proches. Le nombre de groupes, $k = 4$, a été choisi par score de silhouette (0,276, le meilleur entre 2 et 7). Ce groupe devient une variable d'entrée de la régression (détail en annexe A).

Variables d'historique (lag)

Pour chaque département, `RATIO_DMA_lag1` reprend le ratio de l'enquête précédente (`groupby('C_DEPT').shift(1)` sur les données triées par année). C'est la traduction directe de la corrélation de 0,91 observée à l'étape 4.

Découpage entraînement / test : par année, pas au hasard

Jeu	Années	Lignes	Rôle
Entraînement	2011 à 2019	501	Le modèle apprend sur ces données
Test	2021 et 2023	198	Jamais vues pendant l'entraînement : simulent le futur

Un découpage aléatoire mettrait des années récentes dans l'entraînement et des années anciennes dans le test : le modèle « verrait le futur ». Le découpage par année reproduit la vraie situation de prédiction. Contrôle inscrit dans le code : aucun couple département × année n'est à la fois dans les deux jeux.

Mise à l'échelle

Les variables numériques sont ramenées entre 0 et 1 (`MinMaxScaler`), sinon la population (des centaines de milliers) écraserait le ratio (quelques centaines). Le scaler est calibré **uniquement sur l'entraînement**. Conséquence attendue : sur le test, certaines valeurs dépassent légèrement 1 (jusqu'à 1,03), parce que 2021 et 2023 atteignent des niveaux jamais vus entre 2011 et 2019. Calibrer sur toutes les données aurait masqué ce signal et fait fuiter des informations du test.

ÉTAPE 9 · 16 septembre 2026

9. Établir la baseline

Avant d'entraîner le moindre modèle, il faut un **repère** : sans lui, impossible de savoir si un score est bon. Un R^2 de 0,70 peut être excellent... ou ne rien valoir si une règle triviale fait aussi bien.

Baseline	Règle	Rôle
Naïve	Prédire le ratio de l'enquête précédente, tel quel	Le vrai repère à battre : la corrélation de 0,91 le rend déjà très fort
Moyenne	Prédire toujours la moyenne de l'entraînement (DummyRegressor)	Plancher absolu : un modèle qui fait moins bien n'a rien appris. Ajoutée à l'étape 11

Sur le test (2021 et 2023), la baseline naïve obtient un **R^2 de 0,702** et se trompe en moyenne de **36,5 kg par habitant** (MAE). La baseline moyenne obtient un R^2 de 0,010 : elle n'explique rien.

Les métriques utilisées

Pour chaque ligne, l'erreur est l'écart entre la valeur réelle y et la valeur prédite $\hat{y}$. On résume les n erreurs de trois façons :

Métrique	Formule	Ce qu'elle dit
MAE	$MAE = (1/n) \times \sum y - \hat{y} $	De combien de kg/hab la prédiction se trompe en moyenne. La plus lisible.
RMSE	$RMSE = \sqrt{(1/n) \times \sum (y - \hat{y})^2}$	Même unité, mais les grosses erreurs pèsent plus lourd (elles sont mises au carré).
R^2	$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$ $SS_{res} = \sum (y - \hat{y})^2$ $SS_{tot} = \sum (y - \bar{y})^2$	Part des écarts entre départements expliquée. 1 = parfait ; 0 = pas mieux que prédire la moyenne $\bar{y}$; négatif = pire que la moyenne.

Le calcul complet, pas à pas, pour chaque modèle, pli et année, est donné en annexe B.

ÉTAPE 10 · 16 septembre 2026

10. Premiers modèles

Deux modèles, du plus simple au plus élaboré, entraînés sur les mêmes 4 variables (script 06_train_model.py) :

- **Régression linéaire** : une somme pondérée des variables. Simple et interprétable.
- **Random Forest** : 300 arbres de décision (profondeur maximale 6). Capte les relations non linéaires et mesure l'importance de chaque variable.

Modèle (test 2021-2023)	MAE	RMSE	R ²
Baseline naïve	36,49	44,58	0,702
Régression linéaire	35,89	43,64	0,715
Random Forest	36,05	44,21	0,707

Premier constat : les modèles battent à peine la baseline. La Random Forest attribue **94,9 %** de l'importance au ratio de l'enquête précédente. La population, le profil de traitement et le tonnage précédent se partagent le reste.

Plus de données aiderait-il ?

Nous avons réentraîné la régression sur des portions croissantes de l'entraînement (de 50 à 501 lignes), en mesurant à chaque fois le score sur le même test.

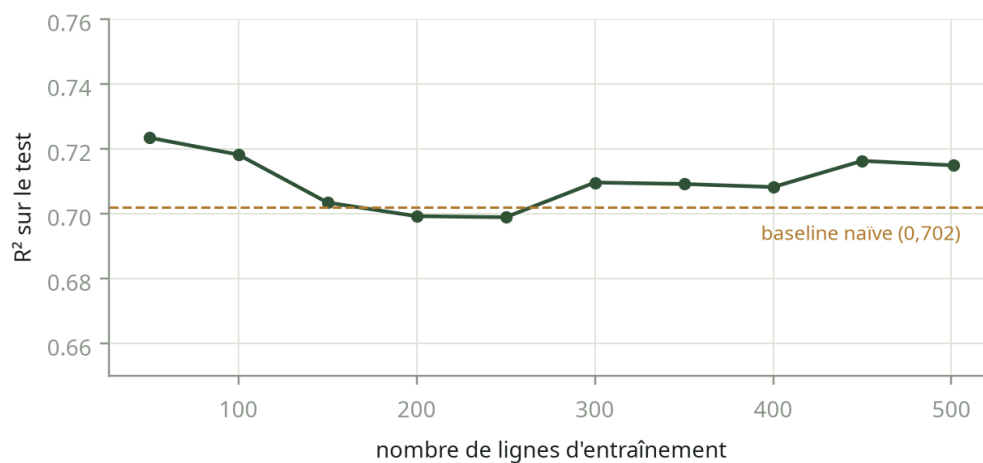


Figure 2 — Courbe d'apprentissage de la régression linéaire : le score ne progresse pas avec le nombre de lignes.

La courbe est **plate dès 50 lignes**. Si le manque de données était le problème, le score monterait avec le nombre d'exemples. Ici, la limite vient du **contenu des variables**, pas de leur quantité.

Une variable supplémentaire : le revenu

Hypothèse : le pouvoir d'achat influence la quantité de déchets. Le revenu médian INSEE a été ajouté, avec son historique (script `08_income_experiment.py`, période 2015–2021 commune aux deux sources) :

Période 2015–2021	MAE	R ²
Baseline naïve	21,72	0,871
Régression sans revenu	19,86	0,887
Régression avec revenu	20,17	0,885

Le revenu **n'améliore rien** (il pèse 2,7 % dans la Random Forest). Hypothèse testée et écartée. Ces scores ne sont pas comparables aux précédents : la période est différente.

ÉTAPE 11 · 24 septembre 2026

11. Preprocessor et pipeline

Les scripts 05 et 06 faisaient chaque transformation à la main. Le script 09_pipeline.py regroupe la même chaîne dans les deux objets de scikit-learn prévus pour ça.

Le preprocessor : l'étape de transformation

Un `ColumnTransformer` applique à chaque type de colonne la transformation adaptée :

- variables numériques `MinMaxScaler` (entre 0 et 1);
- profil de traitement `OneHotEncoder` : le cluster (0 à 3) devient 4 colonnes 0/1.

Un défaut corrigé au passage

Dans la première version, le cluster était mis à l'échelle comme un nombre. Or « cluster 3 » n'est pas « trois fois cluster 1 » : c'est une catégorie. Le traiter comme un nombre lui donnait un faux ordre. L'encodage one-hot corrige ce point. L'effet sur le score est quasi nul (R^2 de 0,715 à 0,716), ce qui est cohérent avec le poids très faible du cluster.

La pipeline : la chaîne complète

```
preprocessor = ColumnTransformer([
    ("num", MinMaxScaler(), ["VA_POPANNEE", "RATIO_DMA_lag1", "TONNAGE_DMA_lag1"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["cluster"]),
])
pipeline = Pipeline([("preprocessor", preprocessor), ("modele", LinearRegression())])

pipeline.fit(X_train, y_train)    # calibre le preprocessor PUIS entraîne, sur le train seul
pipeline.predict(X_test)         # mêmes transformations PUIS prédiction
```

Ce que la pipeline apporte :

- **Aucune fuite de données, par construction** : le preprocessor est calibré à l'intérieur de `fit`, donc sur l'entraînement seul, y compris dans chaque pli de la validation croisée. Ce n'est plus une règle à respecter à la main.
- **Le même traitement partout** : test et nouvelles données passent exactement par les mêmes transformations.
- **Une comparaison équitable** : tous les modèles reçoivent le même preprocessor, seul le modèle change.
- **Un seul objet** prend les données brutes et renvoie une prédiction.

ÉTAPE 12 • 24 septembre 2026

12. Évaluation et interprétation

Deux évaluations complémentaires :

- une **validation croisée temporelle** sur l'entraînement : 4 plis où l'on entraîne sur les enquêtes passées et l'on valide sur la suivante (2013, 2015, 2017, 2019) ;
- le **test final** sur 2021 et 2023, jamais vus.

Modèle	Validation croisée : MAE	R ²	Test : MAE	RMSE	R ²
Baseline moyenne	62,23	0,013	61,99	82,14	0,010
Baseline naïve	18,68	0,872	36,49	44,58	0,702
Régression linéaire	19,55	0,865	35,75	43,53	0,716
Random Forest	20,88	0,845	36,08	44,36	0,705

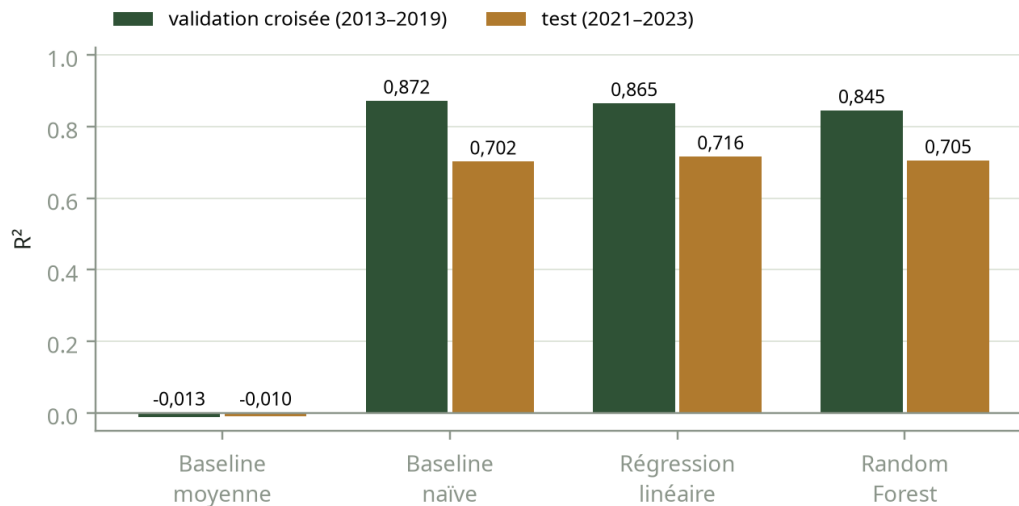


Figure 3 — R² des quatre approches, en validation croisée et sur le test.

D'où viennent ces chiffres : le calcul sur le test

Les 198 erreurs du test ($y - \hat{y}$) sont additionnées en valeur absolue pour la MAE, au carré pour la RMSE et le R^2 . SS_{tot} est la même pour tous les modèles : c'est la variabilité des vraies valeurs autour de leur moyenne (541,3 kg/hab).

Modèle	$\sum y - \hat{y} $	MAE = $\sum / 198$	$\sum (y - \hat{y})^2$	RMSE = $(\sum / 198)$	$R^2 = 1 - \sum / SS_{\text{tot}}$
Baseline moyenne	12 274,2	61,99	1 336 053	82,14	$1 - 1\,336\,053 / 1\,322\,733 = -0,010$
Baseline naïve	7 224,8	36,49	393 520	44,58	$1 - 393\,520 / 1\,322\,733 = 0,702$
Régression linéaire	7 078,8	35,75	375 240	43,53	$1 - 375\,240 / 1\,322\,733 = 0,716$
Random Forest	7 144,6	36,08	389 706	44,36	$1 - 389\,706 / 1\,322\,733 = 0,705$

Le gain sur la baseline naïve est-il réel ?

Sur le test, la régression linéaire se trompe de 0,74 kg/hab de moins que la baseline naïve. Un bootstrap (5 000 rééchantillonnages des 198 départements-années) donne un intervalle de confiance à 95 % de **0,14 à 1,34 kg/hab** : réel au sens statistique, mais **négligeable** (2 % de l'erreur, sur un ratio moyen d'environ 520 kg/hab). La régression ne fait mieux que la baseline que sur 54 % des départements-années. Elle a d'ailleurs réappris la persistance : sur la seule variable d'historique, elle donne **ratio 0,97 × ratio précédent + 17**.

Modèle retenu

Aucun modèle ne bat la baseline naïve de façon utile : elle est la meilleure en validation croisée, et la régression linéaire ne la devance que de très peu sur le test. S'il faut retenir un modèle entraîné, c'est la **régression linéaire** : meilleure que la Random Forest sur les deux évaluations, plus simple et interprétable. Mais la conclusion honnête est que la règle « comme à l'enquête précédente » fait aussi bien.

Pourquoi le score baisse-t-il sur le test ?

Le R^2 passe de 0,865 en validation croisée à 0,716 sur le test. Ce n'est **pas du surapprentissage** : la baseline naïve, qui n'apprend rien, chute de la même façon.

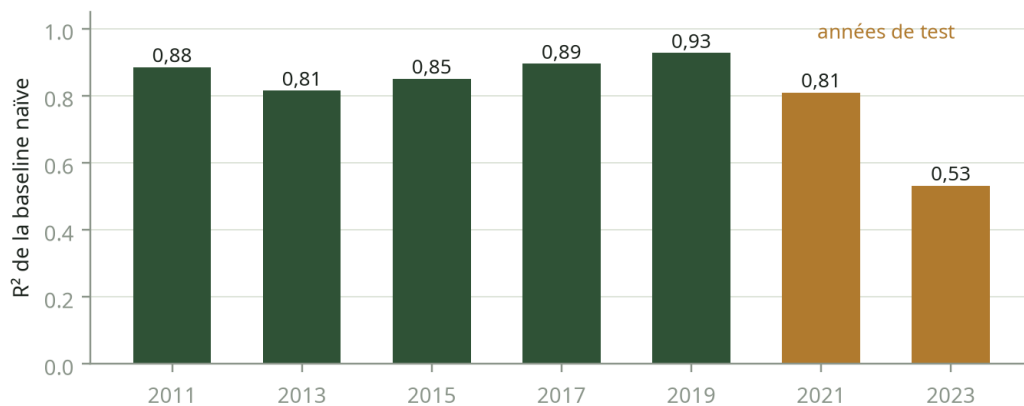


Figure 4 — Qualité de la règle « comme à l'enquête précédente », année par année.

Les années de test marquent une **rupture** : le ratio moyen monte à 563 kg/hab en 2021 (contre 525 à 540 auparavant), puis retombe à 519 en 2023. « Comme la dernière fois » devient une moins bonne hypothèse, et le modèle, qui repose surtout dessus, baisse avec elle. L'origine de cette rupture n'est pas identifiable avec nos données. La période Covid et l'extension des consignes de tri sont des hypothèses, non vérifiées.

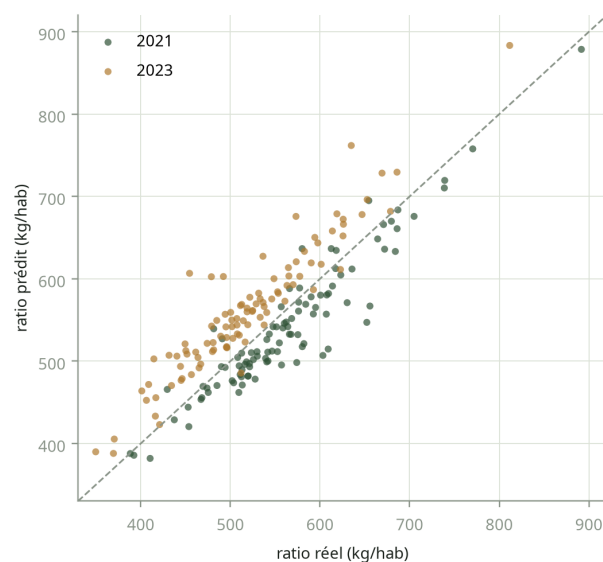


Figure 5 — Ratio prédit par la pipeline (régression linéaire) contre ratio réel, jeu de test. En 2023, la plupart des points passent sous la diagonale : le modèle prévoit plus de déchets qu'il n'y en a eu.

ÉTAPE 13 · 24 septembre 2026

13. Tester le modèle et prédire 2025

Une vérification « terrain » a déjà eu lieu

Nous ne pouvions pas aller mesurer nous-mêmes les déchets des départements. Mais le découpage par année joue exactement ce rôle : le modèle a été entraîné jusqu'en 2019, puis confronté aux **vraies** valeurs de 2021 et 2023, publiées par l'ADEME et qu'il n'avait jamais vues. C'est un test en conditions réelles, a posteriori (on parle de *backtest*). Les scores du chapitre 12 sont donc mesurés sur le terrain.

Et des données simulées (mocks) ?

Générer de fausses données reste utile pour **tester le code** : vérifier que la pipeline accepte un département inconnu, une valeur manquante ou un nouveau format sans planter. En revanche, elles ne peuvent pas servir à **évaluer le modèle** : c'est nous qui fixerions les règles qui les produisent, et le modèle serait jugé sur sa capacité à retrouver nos propres hypothèses. Le score serait circulaire. Nous n'avons donc utilisé que des données réelles pour mesurer la performance.

Prédire l'enquête 2025

L'enquête 2025 n'est pas encore publiée : c'est une vraie prédiction (script `12_prediction_2025.py`). Le modèle retenu est réentraîné sur **toutes** les enquêtes de 2011 à 2023 (699 lignes), puis reçoit, pour chaque département, les valeurs 2023 comme « enquête précédente ». La population 2025 n'étant pas connue, celle de 2023 est utilisée (approximation).

L'intervalle vient des erreurs réellement observées sur le test : 80 % d'entre elles étaient inférieures à **50,4 kg/hab**. Chaque prédiction est donc donnée à $\pm 50,4$ kg/hab.

Département	Réel 2023	Prédit 2025	Intervalle (80 %)
Ain	463,9	467,2	416,8 – 517,6
Gironde	519,4	511,6	461,2 – 562,0
Nord	524,9	511,8	461,4 – 562,2
Paris	421,0	409,6	359,2 – 460,0
Landes	811,6	790,5	740,1 – 840,9
Cotes-d'Armor	685,9	668,7	618,3 – 719,1
Hauts-de-seine	349,7	345,7	295,3 – 396,1
Rhône	370,4	361,6	311,2 – 412,0

En moyenne, le modèle prévoit **518,0 kg/hab** en 2025, contre 519,4 en 2023 : il prolonge la situation actuelle. Toutes les prédictions sont dans `data/processed/predictions_2025.csv`. Elles pourront être vérifiées à la publication de l'enquête 2025. Limite connue : si la France entière bouge à nouveau d'un bloc, comme entre 2021 et 2023, le modèle ne pourra pas l'anticiper (chapitre 14).

ÉTAPE 14 · 24 septembre 2026

14. Comment améliorer le modèle

Pour savoir quelles informations ajouter, nous avons d'abord regardé **où** le modèle se trompe (script `11_analyse_erreurs.py`).

L'erreur est surtout commune à tous les départements

Année de test	Erreur moyenne du modèle	Écart-type	Évolution réelle moyenne	Départements en baisse
2021	-21,6 kg/hab	27,8	21,5 kg/hab	11%
2023	42,7 kg/hab	27,3	-43,8 kg/hab	97%

En 2021, presque tous les départements ont augmenté ; en 2023, 97% ont baissé. Le modèle, qui prolonge la valeur précédente, sous-estime d'environ 22 kg/hab en 2021 et surestime d'environ 43 kg/hab en 2023, pour **tous** les départements à la fois. Une grande partie de l'erreur ne vient donc pas d'un département en particulier, mais d'un mouvement national.

Pour mesurer ce que vaudrait cette information, une expérience de pensée : ajouter à la valeur précédente de chaque département l'évolution nationale moyenne de l'année (connue ici après coup).

Approche (test 2021-2023)	MAE	R ²
Baseline naïve	36,49	0,702
Régression linéaire (modèle retenu)	35,75	0,716
Baseline naïve + évolution nationale connue	19,89	0,881

Ce que ça montre

Connaître la tendance nationale ferait passer l'erreur moyenne de 36,5 à 19,9 kg/hab, et le R² de 0,70 à 0,88. Aucune variable locale testée (population, profil de traitement, revenu) n'en approche. La piste prioritaire est donc une information qui **change dans le temps**, d'abord au niveau national, puis au niveau du département.

Les plus grosses erreurs

Département	Année	Précédent	Réel	Prédit	Erreur
Eure-et-Loir	2023	609	454	607	152
Corse-du-Sud	2023	770	635	762	127
Territoire-de-Belfort	2023	603	479	603	124
Oise	2023	607	492	603	111
Pas-de-Calais	2021	553	652	547	-105
Haute-Corse	2023	680	573	676	103

Deux motifs se dégagent. Certains départements font un **aller-retour** brutal : l'Eure-et-Loir passe de 513 à 609 puis 454 kg/hab, le Territoire-de-Belfort de 502 à 603 puis 479. Un tel yo-yo évoque plutôt un changement de périmètre ou de déclaration qu'un changement de comportement : c'est à vérifier auprès de la source. Par ailleurs, la **Corse** a l'erreur moyenne la plus forte (+52 kg/hab). Or la population utilisée pour le ratio est la population résidente : les touristes produisent des déchets sans être comptés, ce qui pourrait expliquer les ratios élevés des départements touristiques (Corse, Landes, Var).

Les pistes, par ordre de priorité

Oui, le modèle pourrait être consolidé avec davantage d'informations. Voici celles qui répondent aux erreurs observées. Les sources sont des pistes à confirmer au moment de la collecte.

Priorité	Information à collecter	Pourquoi (hypothèse)	Source envisagée
1 · national, dans le temps	Consommation des ménages ; événements réglementaires (extension des consignes de tri à tous les emballages, généralisée début 2023 ; tri à la source des biodéchets, obligatoire depuis 2024) ; crise sanitaire 2020-2021	L'essentiel de l'erreur est un mouvement commun à tous les départements	INSEE (comptes nationaux) ; textes réglementaires, codés en variables 0/1 par année
2 · département, dans le temps	Tarification incitative (part de la population couverte) ; nombre de déchèteries	Des politiques qui changent la production d'une enquête à l'autre, et à des dates différentes selon les territoires	ADEME / SINOE (annuaire des déchèteries repéré au début du projet)
3 · mode de vie	Tourisme (capacité d'hébergement, résidences secondaires) ; type d'habitat (maisons avec jardin) ; structure par âge, taille des ménages	Population réelle différente de la population résidente ; les jardins produisent des déchets verts	INSEE (recensement, tourisme) ; le fichier par âge déjà téléchargé
4 · plus fin géographiquement	Descendre au niveau des intercommunalités (EPCI), qui organisent la collecte	Des situations plus variées (une EPCI adopte la tarification incitative, sa voisine non)	ADEME, si les indicateurs existent à ce niveau. Le niveau commune n'est pas réaliste : le tonnage n'y est pas mesuré
Qualité	Vérifier les allers-retours suspects (Eure-et-Loir, Territoire-de-Belfort)	Une erreur de déclaration n'est pas un comportement à apprendre	ADEME (métadonnées SINOE)

Plus de lignes, ou plus d'information ?

Descendre à l'échelle des villes ne suffirait pas en soi : la courbe d'apprentissage (chapitre 10) montre qu'avec les mêmes variables, multiplier les lignes n'améliore rien. Un niveau plus fin n'est utile que s'il apporte des **situations différentes**, par exemple des politiques de collecte qui varient d'un territoire à l'autre.

Comment intégrer une nouvelle variable

La démarche est la même pour chaque piste, et la pipeline la rend simple :

- **Collecter et documenter** la source : producteur, années, niveau géographique, licence.
- **Harmoniser** sur la clé département × année, avec les mêmes contrôles (unicité, valide, nombre de lignes avant et après).
- **Ne garder que ce qui est connu avant la date prédite** : la valeur de l'enquête précédente, sinon c'est une fuite.

- **L'ajouter au preprocessor** : une ligne dans la liste des colonnes numériques ou catégorielles. Le reste de la pipeline ne change pas.
- **Évaluer avec le même protocole** : validation croisée temporelle, test 2021–2023, comparaison à la baseline naïve.
- **La garder seulement si le gain est réel et utile** : intervalle de confiance du bootstrap au-dessus de 0, et gain significatif en kg/hab.
- Autre piste de modélisation : prédire directement l'**évolution** (y valeur précédente) plutôt que le niveau, pour que le modèle se concentre sur ce qui change.

Précaution : avec seulement 8 enquêtes, une variable nationale ne prend que 8 valeurs. Le risque de sur-interpréter une coïncidence est réel ; il faudra plusieurs enquêtes supplémentaires pour confirmer un effet.

15. Problèmes rencontrés et conclusion

Problème	Ce que nous avons fait
Même nom de colonne, périmètre différent (gravats)	Cible toujours lue dans le fichier hors gravats
Une enquête tous les deux ans, pas de 2022	Assumé, non imputé ; le temps est porté par l'historique du département
Le tonnage copie la population (fuite)	Cible ramenée par habitant
Doublons de clé invisibles à <code>duplicate()</code>	Unicité vérifiée, jointures validées, lignes comptées
Cluster traité comme un nombre	Encodé en one-hot dans le preprocessor
Face à la moyenne, tout modèle paraît excellent	Comparaison systématique à la baseline naïve, gain testé par bootstrap
Le score stagne	Courbe d'apprentissage : la limite vient des variables, pas du volume
Chute du score sur le test	Analyse année par année : rupture dans les données en 2023
Pas de mesure « terrain » possible	Test sur des années réelles jamais vues ; pas de données simulées pour évaluer
D'où vient l'erreur restante ?	Surtout d'un mouvement national : piste prioritaire pour améliorer le modèle

Conclusion

Le projet aboutit à une chaîne supervisée complète et vérifiée : sources consolidées et contrôlées, cible choisie pour éviter la fuite, entrées X et sortie y clairement séparées, découpage temporel, preprocessor et pipeline, comparaison à deux baselines, validation croisée, test sur des années réelles et prédiction de l'enquête 2025. Son résultat principal est une conclusion sur les données : **la production de déchets par habitant d'un département est avant tout une affaire d'inertie**. Les caractéristiques stables du département (population, politique de traitement, revenu) n'apportent presque rien au-delà de son propre historique. L'analyse des erreurs montre où chercher la suite : des informations qui évoluent dans le temps, d'abord au niveau national.

Suite du module : l'apprentissage non supervisé, dont le clustering de l'annexe A est une première application.

Annexe A. Clustering des profils de traitement

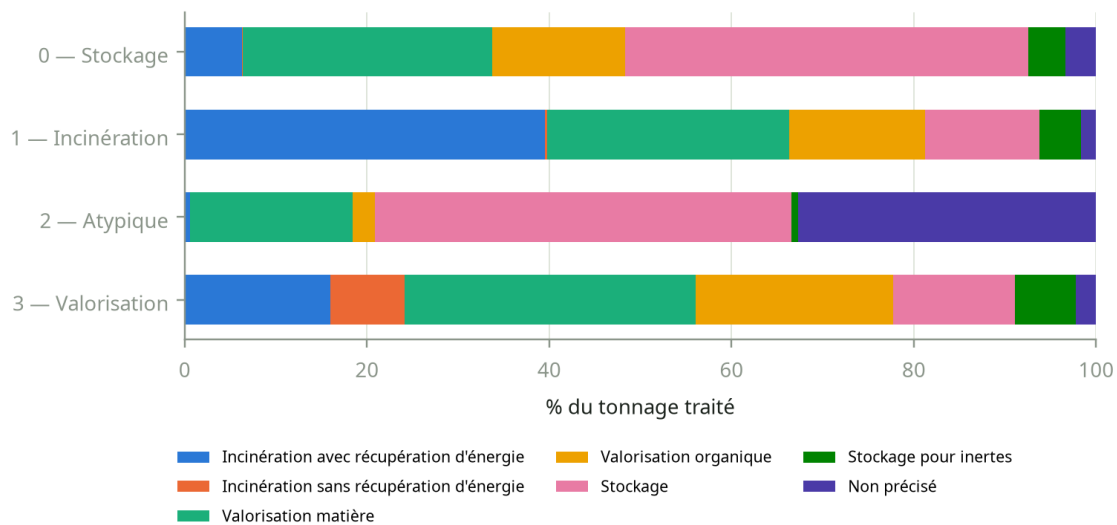


Figure 6 — Profil de traitement moyen de chaque groupe de départements (k-means, k = 4).

Groupe	Profil dominant	Départements
0 — Stockage	44 % de stockage, 27 % de valorisation matière	42, dont Aisne, Allier, Ardèche
1 — Incinération	40 % d'incinération avec récupération d'énergie, environ 6 fois plus que le groupe 0	52, dont Ain, Calvados, Finistère
2 — Atypique	33 % « non précisé » et 46 % de stockage : probablement un défaut de déclaration	2 : Lozère, Mayotte
3 — Valorisation	54 % de valorisation matière et organique, le plus haut des 4 groupes	5 : Charente, Charente-Maritime, Landes, Morbihan, Savoie

Choix du nombre de groupes

Le score de silhouette mesure si chaque département est plus proche de son groupe que du groupe voisin (de 1 à 1). Le meilleur score est obtenu pour k = 4 :

k	2	3	4	5	6	7
Silhouette	0,244	0,228	0,276	0,249	0,241	0,248

Annexe B. Détail des calculs

Tous ces chiffres sont produits par le script `10_detail_metriques.py (make details)`.

B.1 Trois erreurs, une par une (test 2023, régression linéaire)

Département	Ratio 2021 (entrée)	Réel 2023 (y)	Prédit ($\hat{y}$)	Erreur $y - \hat{y}$	$ y - \hat{y} $	$(y - \hat{y})^2$
Ain	503,6	463,9	504,5	-40,6	40,6	1 651
Landes	891,6	811,6	883,4	-71,8	71,8	5 153
Paris	429,6	421,0	422,9	-1,9	1,9	4

La MAE est la moyenne de la colonne $|y - \hat{y}|$ sur les 198 lignes du test ; la RMSE est la racine de la moyenne de la colonne $(y - \hat{y})^2$. Paris est prédit presque parfaitement (sa valeur bouge peu), les Landes beaucoup moins bien (forte baisse en 2023).

B.2 Validation croisée, pli par pli

Validé sur	Entraîné sur	Modèle	MAE	RMSE	R ²
2013	100 lignes	Baseline naïve	21,22	33,00	0,815
2013	100 lignes	Baseline moyenne	63,84	77,74	-0,029
2013	100 lignes	Régression linéaire	21,00	33,26	0,812
2013	100 lignes	Random Forest	20,96	32,65	0,819
2015	200 lignes	Baseline naïve	18,80	29,06	0,850
2015	200 lignes	Baseline moyenne	59,82	75,34	-0,007
2015	200 lignes	Régression linéaire	18,84	28,47	0,856
2015	200 lignes	Random Forest	21,58	32,88	0,808
2017	300 lignes	Baseline naïve	19,28	27,98	0,895
2017	300 lignes	Baseline moyenne	62,67	86,52	-0,007
2017	300 lignes	Régression linéaire	21,92	30,80	0,872
2017	300 lignes	Random Forest	23,97	35,49	0,831
2019	401 lignes	Baseline naïve	15,43	22,74	0,927
2019	401 lignes	Baseline moyenne	62,62	84,57	-0,011
2019	401 lignes	Régression linéaire	16,42	23,82	0,920
2019	401 lignes	Random Forest	17,00	23,68	0,921
Moyenne		Baseline naïve	18,68	28,20	0,872
Moyenne		Baseline moyenne	62,23	81,04	-0,013
Moyenne		Régression linéaire	19,55	29,09	0,865
Moyenne		Random Forest	20,88	31,17	0,845

B.3 Test, année par année

Modèle	MAE 2021	R ² 2021	MAE 2023	R ² 2023
Baseline moyenne	63,43	-0,138	60,55	-0,032
Baseline naïve	28,06	0,810	44,91	0,532
Régression linéaire	27,92	0,813	43,58	0,560
Random Forest	29,81	0,786	42,35	0,565

B.4 Coefficients de la régression linéaire

Les variables numériques étant ramenées entre 0 et 1, un coefficient se lit comme « l'effet, en kg/hab, de passer du minimum au maximum observé ». Ordonnée à l'origine : 266,9.

Variable (après preprocessor)	Coefficient
num__VA_POPANNEE	-37,84
num__RATIO_DMA_lag1	601,95
num__TONNAGE_DMA_lag1	23,64
cat__cluster_0	1,33
cat__cluster_1	0,18
cat__cluster_2	-9,63
cat__cluster_3	8,12

Le ratio précédent domine (+602 entre le département le plus bas et le plus haut). Réentraînée sur cette seule variable, en kg/hab, la régression donne **ratio = 0,969 × ratio précédent + 16,7** : presque la valeur précédente telle quelle.

B.5 Mise à l'échelle Min-Max : un exemple

Sur l'entraînement, la population va de 76 309 à 2 608 346 habitants. Pour l'Ain en 2019 (652 432 habitants) :

$$x' = (x - \min) / (\max - \min) = (652432 - 76309) / (2608346 - 76309) = 0.228$$

B.6 Bootstrap du gain sur la baseline naïve

On tire au hasard, avec remise, 198 départements-années parmi les 198 du test, et l'on calcule la différence de MAE entre la baseline naïve et la régression linéaire. Répété 5 000 fois, cela donne la distribution du gain. 95 % des tirages donnent un gain entre **0,14 et 1,34 kg/hab** : l'intervalle ne contient pas 0, le gain est donc réel, mais il reste très faible face à une erreur moyenne de 36 kg/hab.

Annexe C. Guide d'utilisation du projet

Le dépôt contient un `Makefile` qui permet de rejouer chaque étape par une commande simple, sans connaître le nom des scripts. Chaque commande lance aussi les étapes dont elle dépend.

Prérequis

- Python 3.12 (ou une version récente de Python 3) ;
- `make` et `git` ;
- aucun téléchargement de données : les fichiers sources sont inclus dans le dépôt.

Installation

```
git clone https://github.com/Rxdy/ML-DMA.git
cd ML-DMA
make installer      # crée .venv et installe les dépendances
make                # affiche la liste des commandes
```

Les commandes

Commande	Script	Ce qu'elle fait
<code>make explorer</code>	01	Exploration : dimensions, valeurs manquantes, doublons
<code>make clustering</code>	02	Profil de traitement et k-means
<code>make preparation</code>	03	Jointure avec le cluster, variables d'historique
<code>make correlation</code>	04	Matrice de corrélation (image dans <code>data/figures/</code>)
<code>make decoupage</code>	05	Découpage train/test et mise à l'échelle (version manuelle)
<code>make modeles</code>	06	Baseline et premiers modèles (version manuelle)
<code>make revenu</code>	08	Test du revenu médian
<code>make pipeline</code>	09	Preprocessor, pipeline, validation croisée, test
<code>make details</code>	10	Détail des calculs de MAE, RMSE et R^2
<code>make erreurs</code>	11	Analyse des erreurs
<code>make prediction</code>	12	Prédiction de l'enquête 2025
<code>make supervise</code>	09 à 12	Tout le volet supervisé
<code>make tout</code>	01 à 12	Rejoue tout le projet
<code>make rapport-pdf</code>	—	Régénère ce rapport

Parcours conseillé pour tester le projet : `make installer`, puis `make supervise`. La console affiche les scores de chaque modèle, le détail des calculs, les plus grosses erreurs et un extrait des prédictions 2025. Les résultats sont aussi enregistrés en CSV dans `dechets/data/processed/`.

Bon à savoir

Le script `07_income_prep.py` n'a pas de commande : il assemble des fichiers INSEE téléchargés à la main, non conservés. Son résultat, `insee_revenu_median_dep_2013_2021.csv`, est inclus dans le dépôt, ce qui suffit à `make revenu`. Le rapport PDF nécessite en plus la bibliothèque `reportlab` (incluse dans `requirements.txt`) et les polices Noto.

Organisation du dépôt

ML-DMA/	
├─ Makefile	commandes du projet
├─ requirements.txt	dépendances Python
├─ JOURNAL.md	journal de bord, au fil de l'eau
└─ dechets/	
├─ data/raw/	données sources, jamais modifiées
├─ data/processed/	fichiers produits par les scripts
├─ data/figures/	graphiques
├─ scripts/	01 à 12
├─ rapport/	ce rapport (PDF et générateur), rapport HTML
└─ COMPTE_RENDU.md	synthèse du volet supervisé